

Spring Batch

What is Spring Batch?

Spring Batch is a lightweight, open-source framework designed for batch processing (executing jobs that handle large amounts of data in bulk).

Typical use cases:

- ETL (Extract, Transform, Load) pipelines
- Report generation
- Data migration
- Data cleansing

Quick Outcomes

- Understand chunk-oriented processing and how **writers** participate in transactions.
- Implement multiple writers: **JdbcBatchItemWriter**, **JpaItemWriter**, **FlatFileItemWriter**, **CompositeItemWriter**, **ClassifierCompositeItemWriter**.
- Add **retry/skip**, job **restartability**, and **idempotent** writes.
- Measure & tune: chunk size, JDBC batching, concurrency.
- Ship a POC app with 4 scenarios (basic → pro) you can run locally.

1) Core Concepts Refresher

1.1 Batch anatomy

- **Job** → one or more **Steps**.
- **Chunk step** → reader → processor → writer in transactions per chunk.
- **ItemWriter** writes a *list* of items (size = chunk).

```
public interface ItemWriter<T> { void write(List<? extends T> items) throws Exception; }
```

1.2 Chunk lifecycle (happy path)

begin TX

read N items → process N items → write N items

commit TX

repeat until completion

1.3 Diagram — Chunk cycle

sequenceDiagram

participant R as ItemReader

participant P as ItemProcessor

participant W as ItemWriter

participant DB as Target (DB/File/MQ)

loop per chunk

R->>R: read items (up to chunkSize)

R->>P: pass items

P->>W: processed items

activate W

W->>DB: batch write

deactivate W

end

2) Writer Types & When to Use Them

2.1 Database writers

(JdbcBatchItemWriter, JpaItemWriter — with examples)

2.2 File writers

(FlatFileItemWriter, StaxEventItemWriter — with examples)

2.3 Composite writers

(CompositeItemWriter, ClassifierCompositeItemWriter — with examples)

2.4 Custom writer

(LoggingItemWriter example)

3) Transactions, Retry/Skip, Restartability

- Transaction per chunk.
- Retry/skip rules with `faultTolerant()`.
- Job restartability with `JobRepository`.

4) Performance Tuning Cheatsheet

- Chunk size
- JDBC batching
- I/O locality
- Concurrency & partitioning
- Memory management with JPA

5) POC Project — Structure & Scenarios

(Project layout, dependencies, schema, sample data, YAML config, readers, processors, writers, 4 job scenarios: basic → pro)

6) Monitoring & Observability

- Batch metadata tables
- Actuator & Micrometer
- SQL batching logs
- KPIs for demo

7) Concurrency & Scaling Options

- Multi-threaded step
- Partitioning
- Remote chunking

8) Error Handling Patterns

- Poison pill, validation, duplicate keys
- SkipListener with bad record writer

9) Test Strategy

- StepScope tests
- Happy path
- Retry scenario
- Restart scenario

10) Running the POC

- Build & run with Spring Boot
- Switch jobs with parameters
- Inspect DB + metrics
- Scale dataset

11) Performance Demo Script

- Baseline → tuning chunk size → multithreading → writer comparison → composite impact

12) Troubleshooting

- Many commits → tune chunk size
- OutOfMemory → flush/clear with JPA
- Deadlocks → retries, chunk size
- Duplicates → idempotent SQL
- CSV header repeats → header callback

13) Appendix — Full Snippets

- JdbcBatchItemWriter
- UPSERT examples (Postgres/MySQL)
- Classifier routing snippet
- Multi-thread safe reader tip

14) Ready-to-Use Checklists

- Build & Run
- Observability
- Performance

15) Detailed Execution Flow (with log samples)

Example log when running jdbcJob with chunk=5:

INFO o.s.batch.core.job.SimpleStepHandler - Executing step: [jdbcStep]

DEBUG o.s.jdbc.core.JdbcTemplate - Executing batch update of 5 statements

INFO o.s.batch.core.step.AbstractStep - Step: [jdbcStep] executed in 00:00:02 with readCount=15, writeCount=15, commitCount=3

Key things to notice:

- **readCount** increments per item read.
- **writeCount** increments in batches of chunk size.
- **commitCount** equals number of chunks.

16) Spring Batch Metadata Tables

- **BATCH_JOB_INSTANCE**: unique job definition + parameters.
- **BATCH_JOB_EXECUTION**: each run attempt.
- **BATCH_STEP_EXECUTION**: per step run.
- **BATCH_JOB_EXECUTION_PARAMS**: parameters used.
- **Usage**: Query BATCH_STEP_EXECUTION to measure read/write counts.

17) Docker Compose Setup (DB + Monitoring)

docker-compose.yml

version: '3.8'

services:

db:

image: postgres:15

environment:

POSTGRES_USER: demo

POSTGRES_PASSWORD: demo

POSTGRES_DB: demo

ports: ["5432:5432"]

adminer:

image: adminer

ports: ["8081:8080"]

prometheus:

image: prom/prometheus

ports: ["9090:9090"]

volumes:

- ./prometheus.yml:/etc/prometheus/prometheus.yml

grafana:

image: grafana/grafana

ports: ["3000:3000"]

prometheus.yml (scrape Spring Boot app)

scrape_configs:

- job_name: 'springbatch'

static_configs:

- targets: ['host.docker.internal:8080']

18) Advanced Writer Examples

REST API Writer

```
public class RestApiItemWriter implements ItemWriter<Person> {  
    private final RestTemplate restTemplate = new RestTemplate();  
  
    @Override  
    public void write(List<? extends Person> items) {  
        items.forEach(p -> restTemplate.postForEntity("http://api.example.com/people", p,  
Void.class));  
    }  
}
```

Kafka Writer

```
@Bean  
KafkaItemWriter<String, Person> kafkaWriter(ProducerFactory<String, Person> pf) {  
    KafkaItemWriter<String, Person> writer = new KafkaItemWriter<>();  
    writer.setProducerFactory(pf);  
    writer.setItemKeyMapper(Person::getEmail);  
    return writer;  
}
```

19) Annotated End-to-End Demo Run

1. Start DB via Docker.
2. Run jdbcJob with 10k rows.
3. Observe console/logs:
 - 20 commits for 500-chunk size.
 - Metrics in /actuator/metrics/spring.batch.step.write.count.
4. Check Postgres table: all rows inserted, no duplicates.
5. Restart job with same params → no new execution (job instance already complete).
6. Restart with new params → job runs again.